

CS151 Intro to Data Structures

Runtime Analysis, Arrays

Announcements

- HW0 due Friday
- My office hours will be Tuesday 3-5pm

Outline

- Review: Classes
- Runtime Complexity
- Our first data structure: Arrays

Warm up

1. Create two classes with the following fields. Use inheritance where appropriate.
 - a. `Box`: `size`
 - b. `LabeledBox`: `size`, `label`
2. Define constructors for both classes
3. Create a method `shrink` that takes an `int n`, and reduces the size by `n`. If `n` is `< 0`, throw a custom `ShrinkException`

Warmup 2

What is printed?

```
public class Trace {  
    static int f(int x) throws Exception {  
        if (x < 0) throw new Exception("neg");  
        return x * 2;  
    }  
    public static void main(String[] args) {  
        int[] vals = {3, -1, 5};  
        for (int i = 0; i < vals.length; i++) {  
            try {  
                System.out.println("try " + vals[i]);  
                int r = f(vals[i]);  
                System.out.println("got " + r);  
            } catch (Exception e) {  
                System.out.println("caught " + e.getMessage());  
            }  
        }  
    }  
}
```

Runtime Analysis

- Measuring performance
 - Option 1 (Empirical Runtime Analysis): Compute the time needed to execute the function

```
long startTime = System.currentTimeMillis();  
//code to time  
long endTime = System.currentTimeMillis();  
long elapsed = endTime - startTime;
```

Runtime Analysis

- Option 2 (Theoretical Runtime Analysis): Reason about the runtime in the asymptotic case
- Measured in number of steps. Not time.
- Helps us understand how the runtime of an algorithm grows ***as the input size increases.***
- Written in Big-O notation
- Simplified to keep the dominant term
 - Worst case run time of $2n$ expressed as $O(n)$
 - Worst case run time of $n^2 + 3n$ expressed as $O(n^2)$
- $O(n)$ means the algorithm takes no more than $c \cdot n$

Runtime Analysis

- Big-O notation
 - measures worst case / upper bound
 - Simplified to keep the dominant term
 - Worst case run time of $2n$ expressed as $O(n)$
 - Worst case run time of $n^2 + 3n$ expressed as $O(n^2)$

Runtime Analysis

```
int n = Integer.parseInt(args[0]);  
int sum = 0;  
int i = 0;  
  
while (i < n) {  
    sum = sum + i;  
    i++;  
}  
  
System.out.println(sum);
```

Analyze the runtime of this program.

Answer in Big-O notation.

Constant Time Operations

What are some operations that are independent of the input size?

- Assignments
- Declarations
- Accessing an index in an array
- `arr.length;`
- `mul`, `divide`, `sub`, `add`, `mod`, etc
- `greater than`, `less than`, etc
- `printing`
- `if (x > 100)`

Runtime Analysis Example 2

```
int n = Integer.parseInt(args[0]);
int tot = 0;
int sum = 0;
int i = 0;

while (i < n) {
    tot = tot * i;
    i++;
}

i=0;
while (i < n) {
    sum = sum + i;
    i++;
}
```

Analyze the runtime of this program.

Answer in Big-O notation.

Runtime Analysis Example 3

```
int n = Integer.parseInt(args[0]);  
  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.println(i, j);  
    }  
}
```

Analyze the runtime of this program.

Answer in Big-O notation.

Runtime Analysis Example 4

```
int[] lst =  
    {1,2,3,5,7,12,19,34,55,67,99,101};  
  
int n = lst.length;  
int mid = floor(n/2);  
System.out.println(lst[mid]);
```

Analyze the runtime of this program.

Answer in Big-O notation.

Runtime Analysis Example 5

```
int n = Integer.parseInt(args[0]);  
while (n > 1) {  
    println(n);  
    n = n/2;  
}
```

Analyze the runtime of this program.

Answer in Big-O notation.

Runtime Analysis Example 6

```
int n = Integer.parseInt(args[0]);  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < 10; j++) {  
        println(i + j);  
    }  
}
```

Analyze the runtime of this program.

Answer in Big-O notation.

Runtime Analysis Example 7

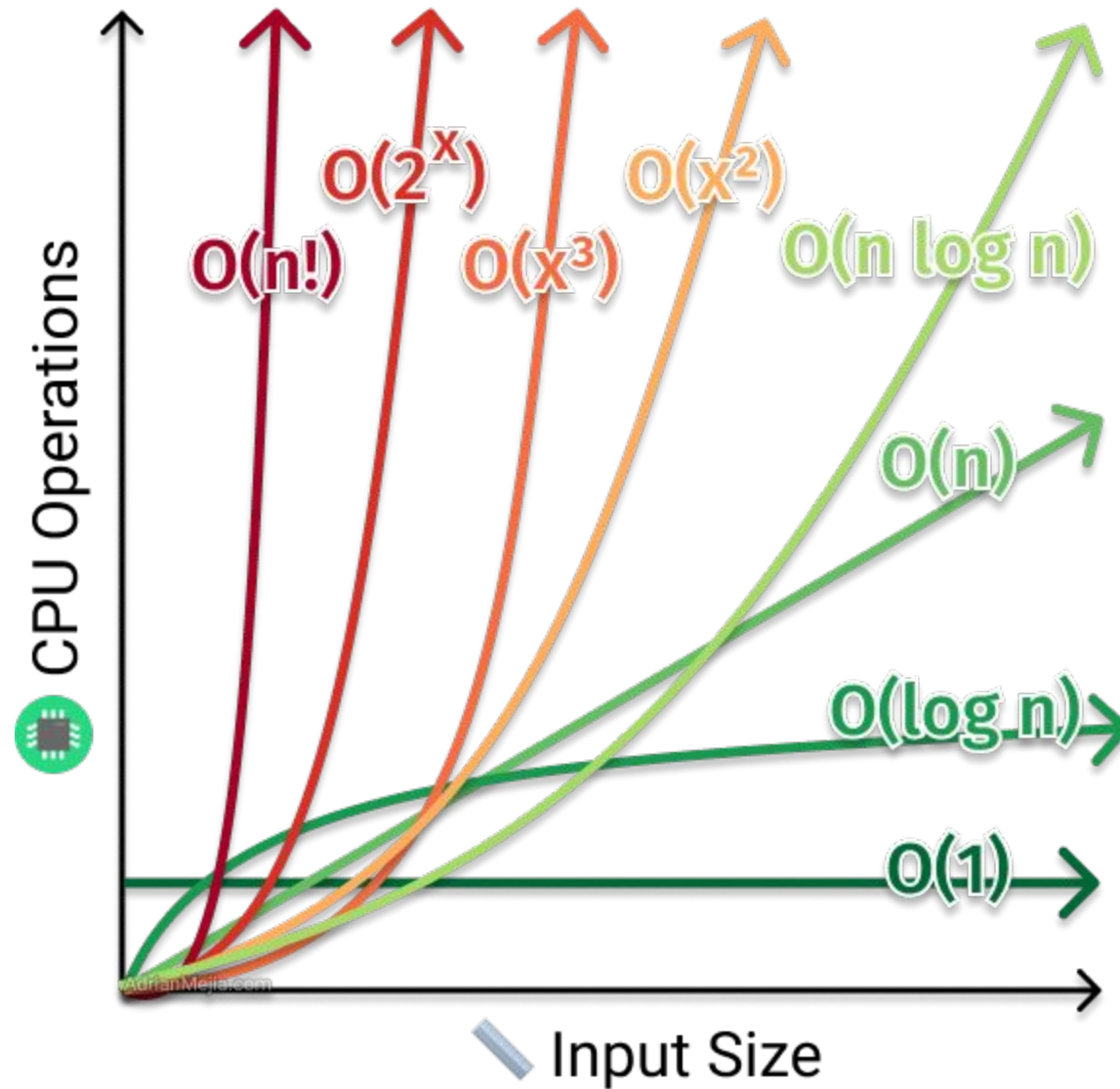
```
int k;  
int n = Integer.parseInt(args[0]);  
for (int i = 0; i < n; i++) {  
    k = n;  
    while (k > 1) {  
        System.out.println(i, k);  
        k = k/2;  
    }  
}
```

Analyze the runtime of this program.

Answer in Big-O notation.



Time Complexity



Runtime Analysis Example 8

```
File f = new File("input.txt");
Scanner sc = new Scanner(f);

while (sc.hasNextLine()) {
    String line = sc.nextLine();
    String[] entries = line.split(" ");

    ...
}
```

Analyze the runtime of this program.

Answer in Big-O notation.

ARRAYS

Arrays

- Homogeneous types
- Contiguous memory
- Fast access
- Fixed size

Type Hierarchy

- Every subclass object is an instance of its superclass
- A superclass object is NOT an instance of the subclass
 - An animal is not necessarily a dog, but a dog is an animal.
 - A dog is not necessarily a poodle, but a poodle is a dog.

Object Casting

- Some operations require objects are a certain type. In order to apply those operations, we can *cast* an object to the required type.
 - Types.java
- Down casting - casting an object of a **parent class** type to an object of a more specific **child class** type
 - Dangerous!!

```
LabeledBox lb = (LabeledBox) box; //ClassCastException!
```

Object Casting

- Does downcasting always cause a ClassCastException?

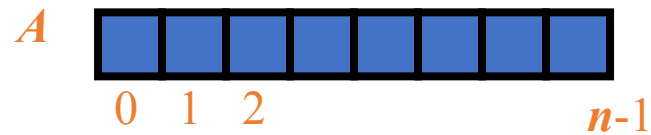
```
Box b = new LabeledBox(10, "label1");  
LabeledBox lb = (LabeledBox) b;
```

Arrays

- Homogeneous types
- Contiguous memory
- Fast access
- **Fixed size**

Let's design an array that can change size!

Imagine we have n items in our array

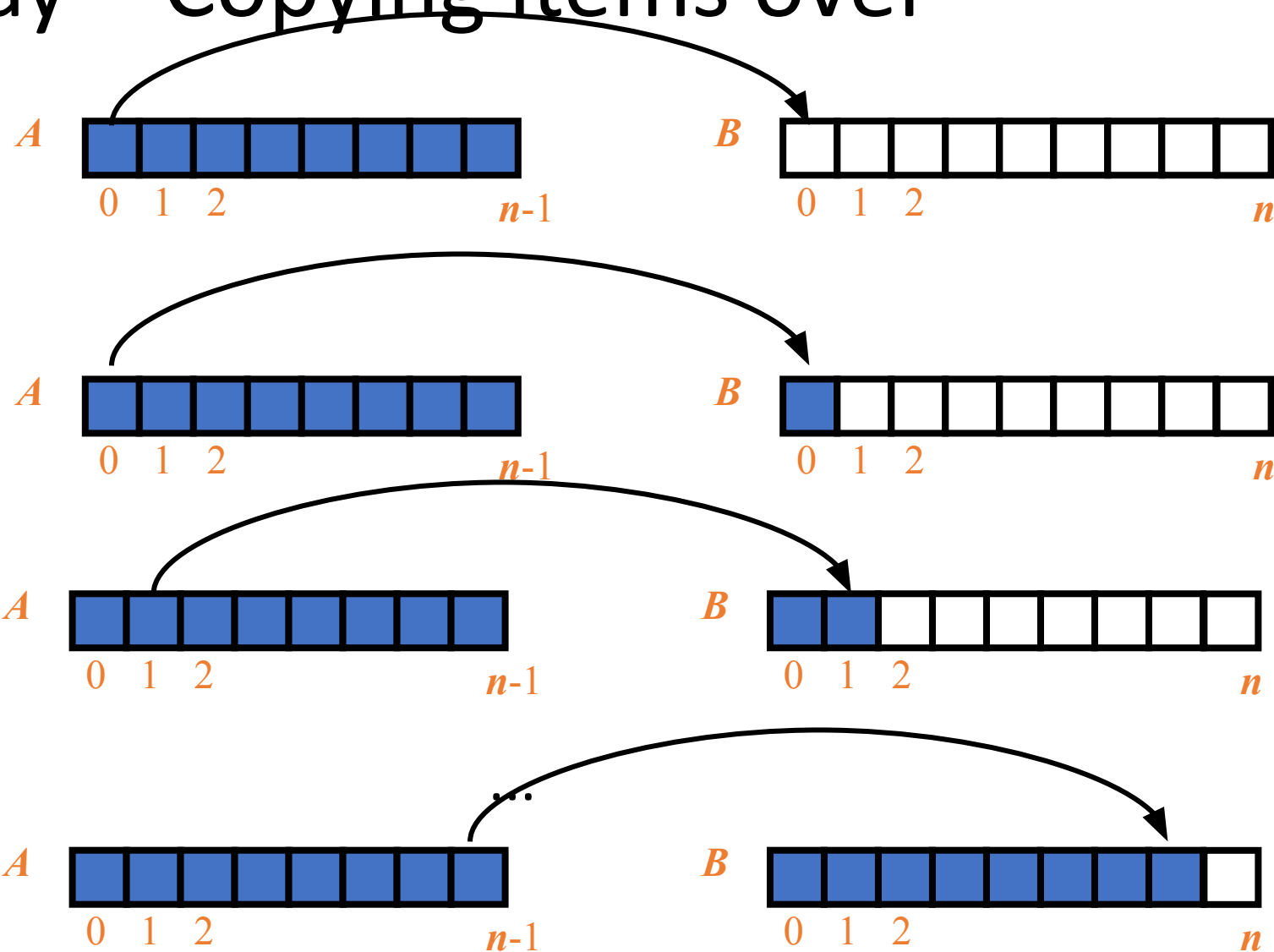


Say we want to add another item, are we stuck?

- No, make a new array and copy all the items over



Array – Copying items over



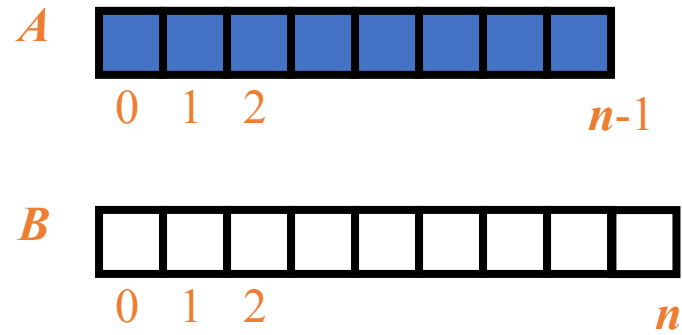
Array Copying

runtime analysis?

$O(n)$

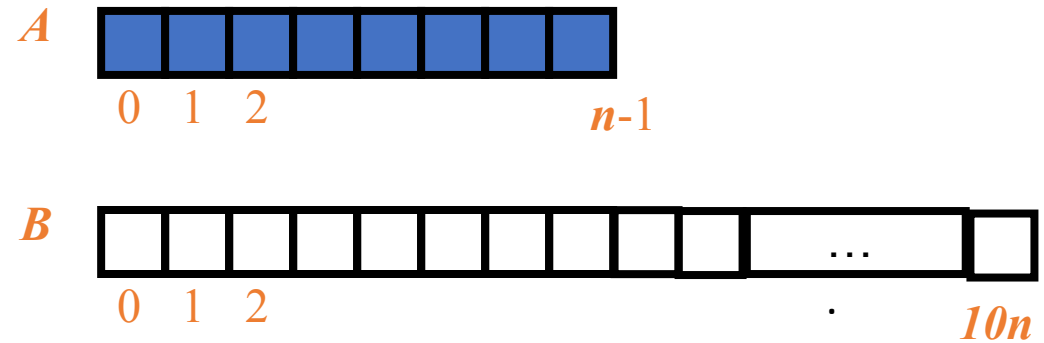
How big should the new array be?

Just one more slot?



Pro: only use much space needed
Con: can lead to lots of copying over

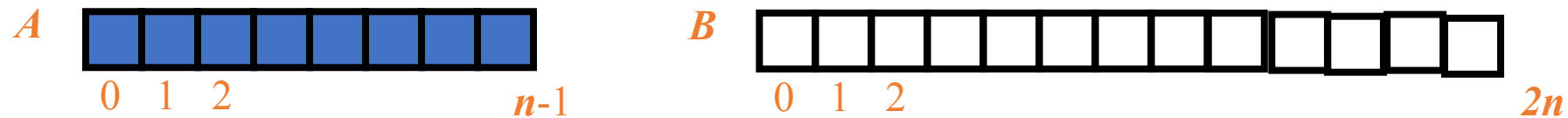
10x the amount of slots?



Pro: don't need to copy lots of times
Con: lots of unused space

How big should the new array be?

- 2 times the length of the full array

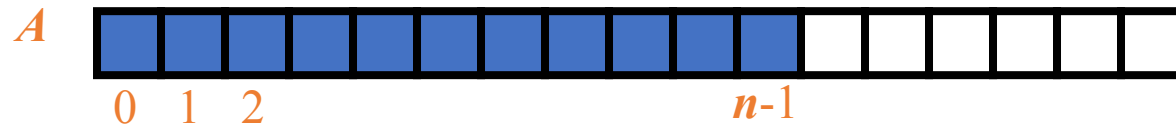


- Compromise between creating too much unnecessary space and having to expand the array too many times

Array Operations

- Insertion
- Removal

Insertion



Where would be the easiest place to insert a new item?
The first open spot?

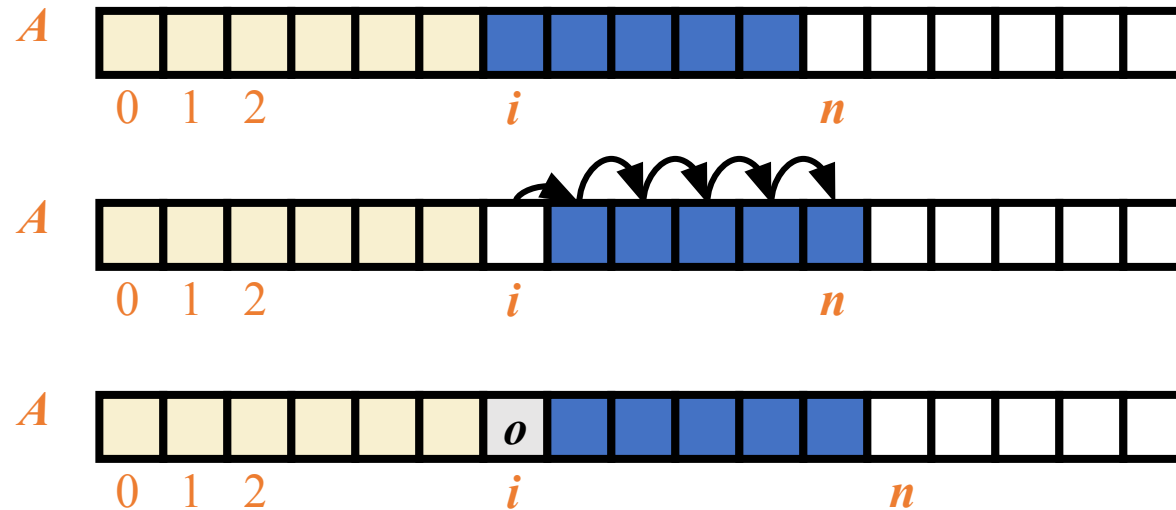


beginning of the array?

If we are going to search for that item a bunch

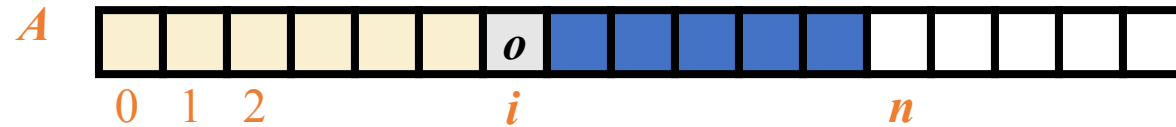
Insertion

- In an operation `insert(i, o)`, we make room for the new element *o* by shifting forward the elements $A[i], \dots, A[n - 1]$



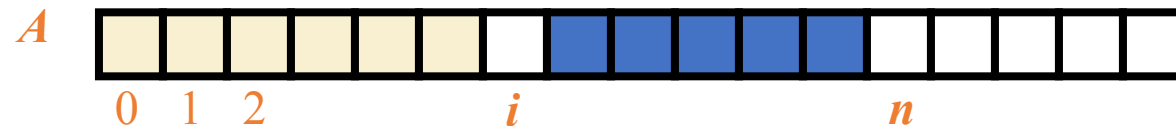
Removal

Say we want to remove the item at index i ?

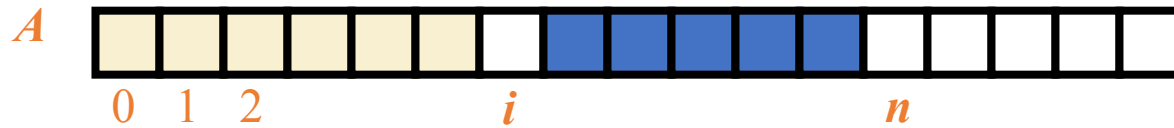


What's the simplest approach?

Just remove it, leaving an empty index



What is wrong with this setup?



Why is having an empty slot in the middle of the array not ideal? What issues might arise?

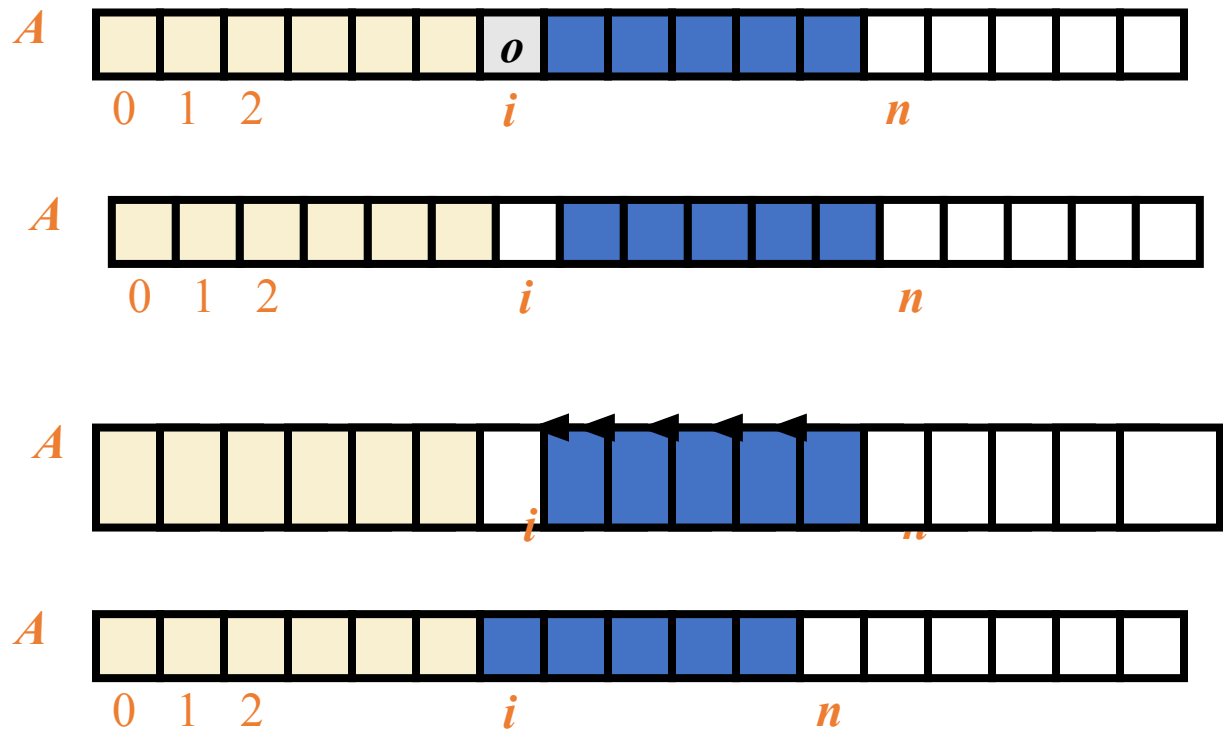
- Makes inserting complicated
 - Where would we put a new item? At the end, or fill the spot?
- Makes looping through the array complicated
 - Need to check for null spots

Removing

In an operation `remove(i)`, we

- remove the element at location i
- then fill the hole by shifting backwards elements

$A[i+1], \dots, A[n-1]$



Array Review

We designed an array that can change size

Insertion:

- Expand array 2x each time it's full
- copy all elements over
- Complexity?

Deletion:

- Fill the hole by shifting everything backwards
- Complexity?

ExpandableArray

We designed an expandable array

Now we will Implement it

In this course, we will use simple Java data structures as an underlying tool to build off.

Let's start coding it! :)

Summary

- **HW0 due Friday**
- Runtime analysis and Big-O notation
 - Worst case
 - Simplify to the dominant term
- Java arrays are *fixed-size*, contiguous, and sequential
- We started building our own ExpandableArray data structure
 - You will finish this in your next Lab / HW